AI Project Phase 1 Report

Group 5
Mani Abedii, Armin Fakhar, Kazem Motealehi

PROBLEM STATEMENT

"The goal of this study is to develop a classification model to predict customer churn, targeting a minimum accuracy of 80% and an F1-score of 0.75."

Why is this a problem?

A business (like a bank, telecom, subscription app, etc.) loses revenue when customers stop using its services. This is called **customer churn**. This brings the question:

"Can we predict which customers are likely to leave soon, before they actually leave?"

Why it matters

Acquiring a new customer is usually **more expensive** than keeping an existing one.

If we can identify "high-risk" customers early, the company can take actions such as: discounts / offers, better support, personalized products, fixing service issues, etc.

So churn prediction is a **decision-support** problem: it helps decide *who to focus on*.

In churn, mistakes have different costs:

- False Negative (FN): customer was going to leave but model says "safe"
  → you miss the chance to retain them (often costly).
- False Positive (FP): model flags a customer as at-risk but they wouldn't leave
  → you might waste money/time on retention offers.

Because of this, the "best" model depends on business priorities:

- If losing customers is very expensive → prioritize Recall (catch as many churners as possible)
- If retention offers are expensive → prioritize Precision (only contact truly at-risk customers)
- In this project we tend to balance both using F1-score.

DATASET INTRODUCTION

Bank_Customers.csv is the dataset we are working with.

- Task: Binary Classification (predict churn)
- Target Label: *Exited*
    - 0 = customer stayed
    - 1 = customer exited (churned)

This Dataset:

contains 10,000 observations (rows) each with 14 columns (features). Although the first 3 columns are IDs and not predictive so dropped.

- Input features used: 10
    - Numerical (6): CreditScore, Age, Tenure, Balance, NumOfProducts
    - Categorial (4): Geography, Gender, HasCrCard, IsActiveMember
- Data type: Tabular (structured numeric + categorial features)

Problems with this dataset:

1. Missing values (around ~5-6% in many features)
2. Mixed feature types
3. Different scaled
4. Potential outliers
5. Label imbalance
    - Stayed = 0: 7963 (79.63%)
    - Exited = 1: 2037 (20.37%)

    A dumb model that predicts always 0 would already get ~80% accuracy, so accuracy alone is misleading. F1-score + ROC should be reported.

EDA charts & data visualizations are further provided in the project.

DATA PREPROCESSING

Preprocessing of our dataset is done in 4 steps:

1) ***Train/test split***
    - **What:** Splits the data into training (80%) and testing (20%).
    - **Why:** Train the model on `X_train/y_train`, and evaluate fairly on unseen data `X_test/y_test.`
    - **How:** Done using the `train_test_split` function from the `scikit-learn` library.

2) ***Handling missing values (imputation)***
    - **When:** Done after splitting the data to avoid leaking information from test into train.
    - **What:**
        - Median imputation for numerical columns by replacing `NaN` in the numeric column with the median value which is more robust than mean when there are outliers.
        - Most-frequent imputation for categorial columns by replacing missing value in categorical columns with the mode (most common category).
    - **How:** Done using the `fit` & `transform` functions of `SimpleImputer` class of `scikit-learn` library.

3) ***One-hot encoding categorial columns (Geography & Gender)***
    - **What:** Converts categories in columns [1,2] into binary indicator columns by:
        - Male/Female → 1/0; simple label encoder
        - France/Germany/Spain → 100/010/001; one-hot encoding
    - **Why:** making the labels numeric without implying an order.
    - **How:** Done using the `fit & transform` functions of `ColumnTransformer` and `OneHotEncoder` classes of `scikit-learn` library. `fit_transform` learns from `X_train` and transforms it. Then `transform` applies the same learned mapping to `X_test`.

4) ***Feature scaling (standardization)***

- **What:** Convert each feature to roughly:
  - mean ≈ 0
  - Standard deviation ≈ 1
- **How:** Using:

$$z = \frac{x - \mu}{\sigma}$$

Done using the `fit`& `transform` functions of `StandardScaler` class of `scikit-learn` library. `fit_transform` computes mean/std on training data then scales it. `transform` scales test using the same mean/std (no leakage).

MODEL BASELINE

We use a simple feedforward neural network (MLP) as the baseline model for this dataset. The model consists of an input layer, multiple small hidden dense layers with ReLU activation functions, and a sigmoid output for binary classification (churn vs. non-churn).

A neural network is suitable for this task because the dataset contains a mixture of numerical and categorical features, and churn behavior may depend on non-linear relationships and feature interactions (e.g., age with balance, geography with product usage). Unlike linear models, neural networks can learn these complex patterns through their layered structure.

A baseline network (for e.g. a shallow neural network with no regularization) is expected to provide a reasonable starting performance, but due to class imbalance, accuracy alone may be misleading, Therefore, performance will be evaluated using metrics such as F1-score and ROC-AUC in addition to accuracy.

EXPERIMENTAL SETUP

How many models are we talking about?

- 2 completely different models.

What are the hyperparameters?

- Learning rate
- Batch size
- No. of epochs
- No. of hidden layers
- No. of neurons per layer
- Optimizer
- Dropout rate (if applied)

What are the evaluation metrics?

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Loss (Binary cross entropy)

What are we recording as results?

- Model configuration
- Hyperparameters values
- Training & validation metrics per epoch